

patch-hub: A terminal-based tool to streamline Linux kernel patch review

Lorenzo Bertin Salvador¹, David Tadokoro¹, Hannah Harrison¹, and Paulo Meirelles¹

¹ IME, University of São Paulo - USP, Brazil

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Open Journals](#) ↗

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

patch-hub is a terminal-based software written in Rust that aims to streamline one of the key workflows in the Linux kernel development model: reviewing patches. Its main features include browsing the patches of each Linux development mailing list, applying them locally for validation, and also the option to respond to them with a *Reviewed-by* tag.

Beyond its practical value, patch-hub is part of a broader effort to modernize Linux kernel development workflows and mitigate bottlenecks. By simplifying the interaction between reviewers, developers, and patches, the tool not only reduces friction in the review process but also creates opportunities for empirical research on software engineering practices within this ecosystem.

Statement of need

A recent area of interest within the Linux kernel community is the long-term sustainability of its development process ([Tadokoro et al., 2025a](#)). For instance, there is concern that the workload of maintainers is not scaling and presents many challenges, as indicated by many community publications ([Corbet, 2013, 2018, 2021](#); [Dean, 2020](#); [Edge, 2013, 2016, 2018](#); [Vetter, 2017](#)); in particular, there is a mismatch between the volume of submitted patches and the capacity to review and integrate them into the project ([Rigby et al., 2014](#)). This scenario highlights potential bottlenecks in the workflow of maintainers, which may impact the project's growth.

On a broader scope, the Kworkflow (kw) project ([Tadokoro et al., 2025b](#)) is a hub of tools that aim to support the most varied workflows of kernel developers. In this sense, patch-hub is a sub-project of kw, with its dedicated codebase and focus on maintainers by directly addressing the bottlenecks associated with patch review. The tool enhances this workflow, which has traditionally involved fragmented, complex, and poorly standardized tasks. Furthermore, patch-hub can also serve as a central platform for research on the review cycle, enabling both a deeper understanding of the relationships among the actors involved and empirical evaluations of strategies to optimize this part of the development flow.

Introduction

The development of the Linux kernel is a prominent example of a large-scale Free Software project. Dozens of subsystems interconnected in a complex *web of trust* (Figure 1) and thousands of contributors have allowed Linux to continue evolving for decades, making it the foundation of most modern computing systems. The project follows a rigorous review and

integration process before a contribution, also known as a *patch*, can reach the software's end users.

In general, kernel development involves many repetitive tasks, both for contributors who seek to have their code incorporated and for maintainers who must ensure the high quality of the contribution. Among these tasks, we emphasize compiling, running, and testing the Linux kernel, as well as organizing, sending, and responding to patches. In practice, this translates to executing long sequences of verbose commands, which waste considerable time to type, are incredibly error-prone, and must be repeated multiple times throughout the development and review of contributions.

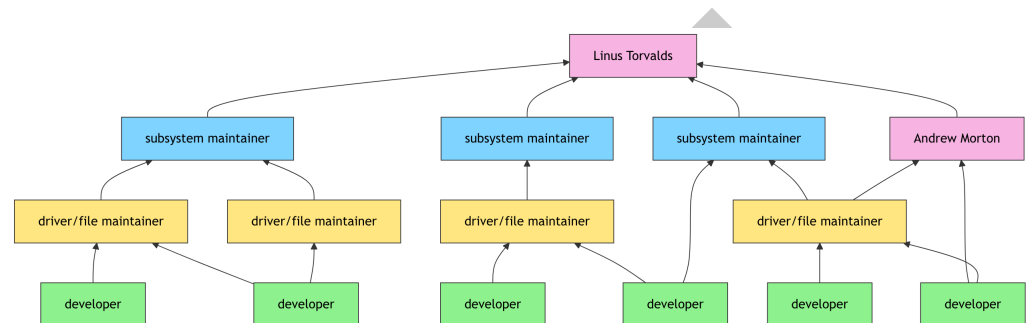


Figure 1: Lifecycle of a Linux patch from sending to merging into the upstream.

Due to the repetitive nature of these tasks, it is common for kernel developers to create or adopt ad hoc scripts to automate such processes, thereby speeding up execution and reducing the likelihood of errors. As a result, this tooling is generally decentralized, leading to duplicated efforts and contributing to the lack of robust standardized solutions for some of these tasks.

A Free Software project that aims to mitigate this issue is *kw*. Written in Bash, it is a tool that helps Linux kernel developers perform various tasks through a unified Command-Line Interface (CLI). Among many other features, users can seamlessly compile and deploy the kernel from source, as well as manage multiple custom configurations for different environments and use cases. In this way, *kw* directly addresses the main bottlenecks arising from repetitive tasks, allowing developers to focus on reviewing and contributing patches themselves.

Within *kw*, another outstanding functionality, which has become an independent utility and is the central topic of this paper, is *patch-hub*. With its dedicated repository and implemented in Rust, *patch-hub* is a Terminal User Interface (TUI) focused on the interaction between developers/maintainers and the *patchsets* (groups of related patches representing a single contribution) representing the development of Linux. Each command in *kw* targets one or more specific tasks, and in the case of *patch-hub*, its goal is to simplify user interaction with mailing lists and the patchsets of each subsystem. Its main features include browsing mailing lists, viewing all patchsets within a list, and interacting with individual patchsets, such as applying one to the local kernel tree or saving a patchset for later analysis.

Under the hood, *patch-hub* leverages *Lore* (lore.kernel.org), the public archive of the Linux development mailing lists, which supports searching for messages and patchsets on demand, in contrast to the traditional model based on subscribing to the lists. Beyond its practical value, *patch-hub* enables empirical investigations into the workflows of maintainers. With appropriate data collection, it could support studies in software engineering aimed at maintaining large-scale projects.

72 **patch-hub**

73 This section presents the core capabilities of patch-hub, highlighting how the software supports
74 the patchset review workflow within kernel development. It then provides an overview of the
75 application's architecture, emphasizing both the design decisions adopted and the way the
76 system integrates with Lore to retrieve and process patchsets.

77 **Features**

78 In general, the main features of patch-hub align with the goal presented in the previous
79 section: to simplify the interaction between those involved in kernel development (specifically
80 maintainers/reviewers) with the patchsets that represent this development.

81 It is worth noting that the project remains in continuous development, and some upcoming
82 features will be exposed in the **Next Steps** section. The following subsections present the most
83 relevant features currently implemented and available.

84 **Integration with Linux development mailing lists**

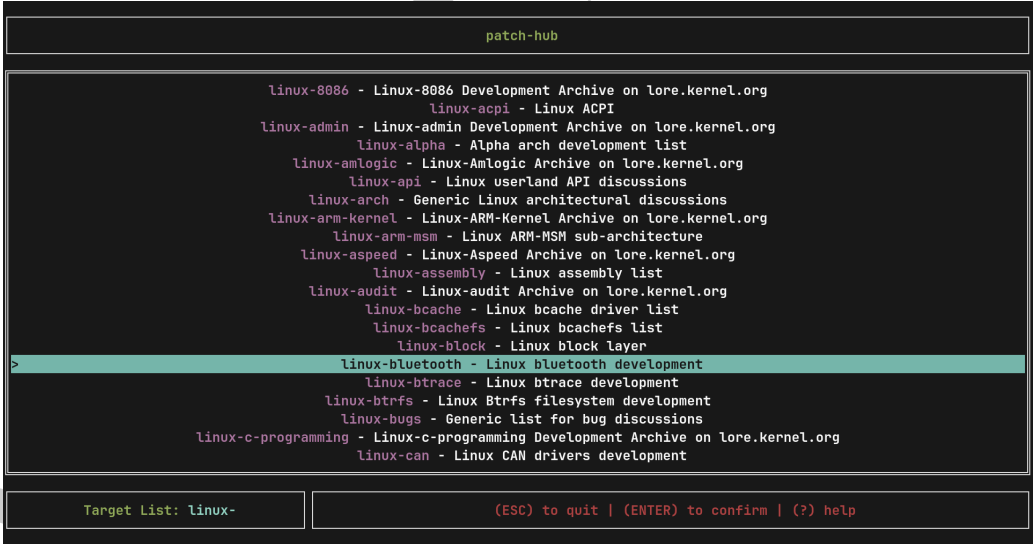


Figure 2: Mailing list selection screen.

85 Users can browse the mailing lists of each subsystem available on lore.kernel.org (Figure 2).
86 For each list, users can navigate through the submitted patches, clustered in their respective
87 patchsets, from most recent to oldest, as shown in Figure 3. Each patchset can be analyzed
88 individually, as described in the next feature.

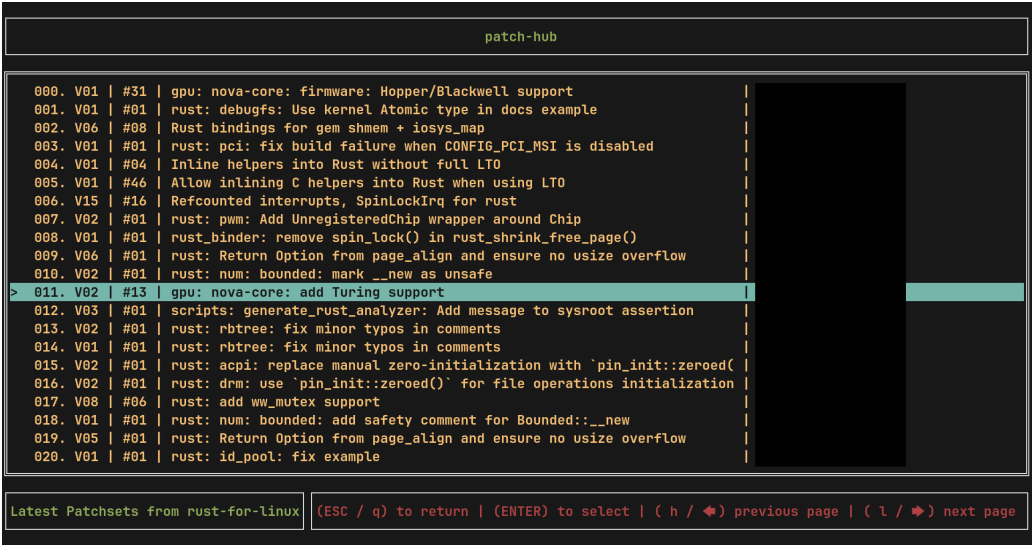


Figure 3: Latest patchsets screen of the rust-for-linux list (redacted patchset authors).

89 **Patchset rendering**

90 For every patchset, users can view its metadata, which includes the title, author, patchset
91 version, and the number of *Reviewed-by*, *Tested-by*, and *Acked-by* tags it has received. Users
92 can also inspect each individual patch within the patchset, as well as the patchset's cover
93 letter. For each patch, the commit message and the *code diff* can be viewed using different
94 renderers for syntax highlighting and colorization. This approach enables users to track the
95 current review state of each patch and conduct their own review efficiently, without the need
96 to set up tools like mutt and perform manual integrations with Lore. This screen is shown in
97 Figure 4.

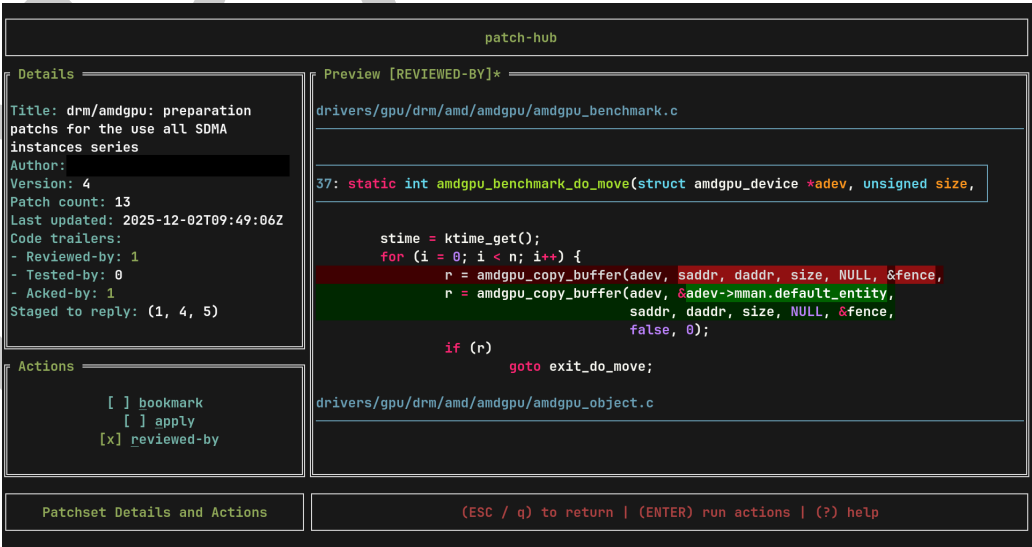


Figure 4: Patchset preview and management screen.

98 **Patchset management**

99 Beyond simply viewing patchsets, users can actively interact with them. Three main actions
100 are supported:

- 101
- 102
- 103
- 104
- 105
1. Bookmark a patchset to access it later.

2. Apply the patchset to a local kernel tree to validate and test the proposed changes (Figure 5).

3. Reply to a patchset (or individual patches in a patchset) with a *Reviewed-by* tag, to indicate endorsement of the contribution.

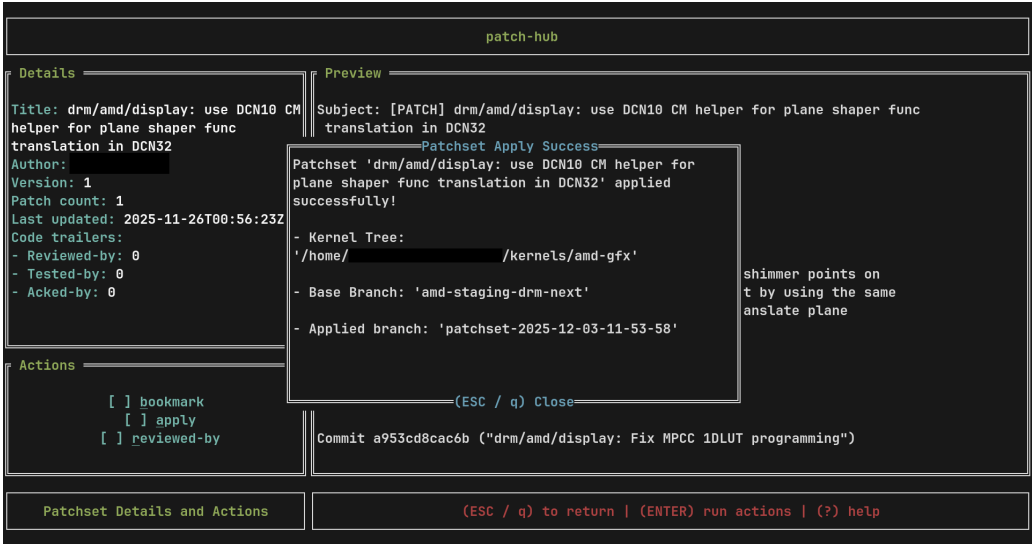


Figure 5: Pop-up after successfully applying patchset.

106

107

108

109

110

111

112

113

Custom configuration

Another important feature is the ability to customize specific system settings within patch-hub itself, as displayed in Figure 6. The main options include (i) selecting the tool to be used for rendering patchsets, (ii) configuring the number of patchsets displayed per page, (iii) defining directories for data and cache storage, and (iv) setting log retention periods. Users can also configure integration with Git commands, such as `git send-email` for replying to patchsets and `git am` for applying a patchset to the local kernel tree. This strategy ensures that the review and application workflow can be tailored to each user's preferences.

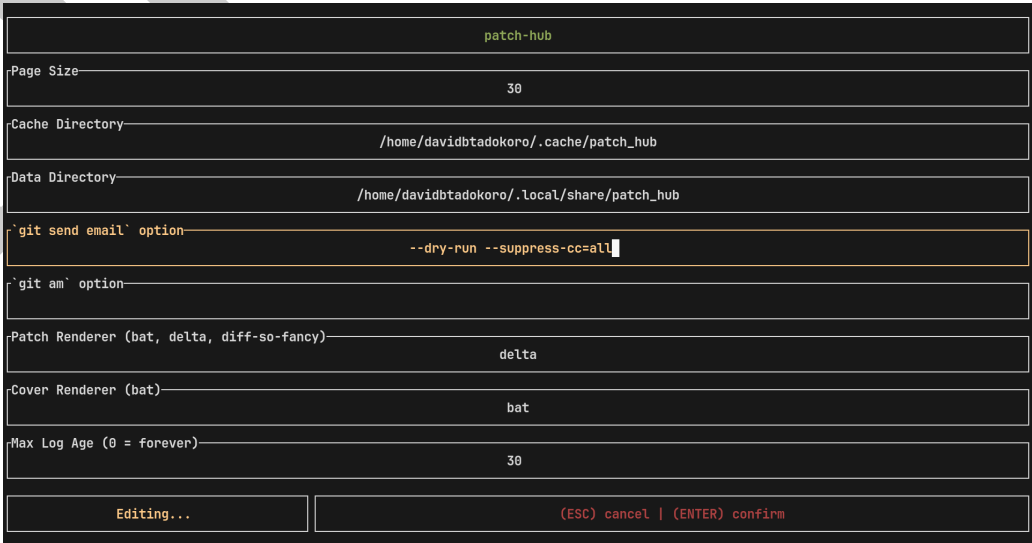


Figure 6: Configuration screen.

114 Architecture

115 The patch-hub architecture contains two fundamental parts:

- 116 1. The core of the application, which handles all state changes triggered by user interaction.
- 117 2. The integration with lore.kernel.org, which provides access to up-to-date patchsets.

118 MVC (Model–View–Controller)

119 We developed the core of the application following the *Model–View–Controller* (MVC) design
120 pattern, where the Model represents the aggregation of all application states, the View
121 represents the abstraction of how those states are rendered to the user, and the Controller
122 manages user interactions and requested actions.

123 Model

124 Practically speaking, patch-hub defines a struct named App (Listing 1), which implements the
125 Model layer. In summary, it contains:

- 126 ■ A CurrentScreen attribute, indicating the application's active screen.
- 127 ■ One struct for each possible screen the user can access (MailingListSelection,
128 BookmarkedPatchsets, LatestPatchsets, DetailsActions, EditConfig).
- 129 ■ A Config struct that stores the current configuration.
- 130 ■ A BlockingLoreAPIClient struct that represents the HTTP client responsible for com-
131 municating with lore.kernel.org.

```
pub struct App {
    pub current_screen: CurrentScreen,
    pub mailing_list_selection: MailingListSelection,
    pub bookmarked_patchsets: BookmarkedPatchsets,
    pub latest_patchsets: Option<LatestPatchsets>,
    pub details_actions: Option<DetailsActions>,
    pub edit_config: Option<EditConfig>,
    pub config: Config,
    pub lore_api_client: BlockingLoreAPIClient,

    /// other less relevant attributes omitted
}
```

132 **Listing 1:** ‘App’ struct snippet.

133 As expected, the Model layer does not handle either end of the application, user interaction,
134 or terminal rendering, but only the core logic of the system. The App struct is responsible for
135 storing each screen’s state, the loaded patchsets, configuration data, and for orchestrating
136 transitions between states. However, it does not directly handle user input or screen rendering.

137 View

138 Since patch-hub is a TUI, the View layer focuses on rendering each screen in the terminal.
139 Concretely, whenever a state change occurs, the terminal is redrawn with the relevant informa-
140 tion for the user, via the draw_ui() function, as shown in Listing 2. This function retrieves
141 the current screen from the App and renders it according to its definition and current state.
142 To draw widgets, patch-hub utilizes the Rust library Ratatui, which provides definitions for
143 colors, alignments, geometric shapes, and other UI components.

144 Besides rendering individual screens, some UI components, such as loading screens and pop-up
145 windows, can appear across multiple views. Their rendering behavior is also handled within
146 the View layer.

```
pub fn draw_ui(f: &mut Frame, app: &App) {
    // beginning of the function omitted

    let chunks = Layout::default()
        .direction(Direction::Vertical)
        .constraints([
            Constraint::Length(3),
            Constraint::Min(1),
            Constraint::Length(3),
        ])
        .split(f.area());

    render_title(f, chunks[0]);

    match app.current_screen {
        CurrentScreen::MailingListSelection => mail_list::render_main(f, app, chunks[1])
        CurrentScreen::BookmarkedPatchsets => {
            bookmarked::render_main(f, &app.bookmarked_patchsets, chunks[1])
        }
        CurrentScreen::LatestPatchsets => latest::render_main(f, app, chunks[1]),
        CurrentScreen::PatchsetDetails => details_actions::render_main(f, app, chunks[1]),
        CurrentScreen::EditConfig => edit_config::render_main(f, app, chunks[1]),
    }

    navigation_bar::render(f, app, chunks[2]);

    /// rest of the function omitted
}
```

147 **Listing 2:** 'draw_ui()' function snippet.

148 The View layer is unaware of how information is stored or which user interactions have led to
 149 the current state. It only needs the current state to decide how to compose and display the
 150 interface elements.

151 **Controller**

152 The Controller layer coordinates the chain of operations triggered by user actions. In general, it
 153 captures keyboard events and routes them to their corresponding actions (illustrated in Listing
 154 3), which typically involve updating the App (Model) and then redrawing the screen (View).
 155 Each screen has its own event handler, and whenever a user action causes a screen transition,
 156 the corresponding handler function is invoked.

157 Thus, the Controller directly interacts with both the Model and the View, orchestrating their
 158 operation at a high level.

```
match key.code {
    KeyCode::Char('?') => {
        let popup = generate_help_popup();
        app.popup = Some(popup);
    }
    KeyCode::Esc | KeyCode::Char('q') => {
        app.reset_latest_patchsets();
        app.set_current_screen(CurrentScreen::MailingListSelection);
    }
    KeyCode::Char('j') | KeyCode::Down => {
        latest_patchsets.select_below_patchset();
    }
}
```



```

    }
    KeyCode::Char('k') | KeyCode::Up => {
        latest_patchsets.select_above_patchset();
    }

```

159 **Listing 3:** Example of Key-to-action routing.

160 Lore API

161 With the MVC structure established, the other central pillar of patch-hub is its integration
 162 module with lore.kernel.org, which enables fetching patchsets. This module is not part of
 163 the MVC structure because it operates independently of the core business logic; it merely
 164 provides data to patch-hub, and could be replaced by another source without requiring changes
 165 elsewhere in the system.

166 The primary goal of this module is to communicate with lore.kernel.org via HTTP requests to
 167 retrieve the list and details of patchsets.

168 The HTTP client is represented by the `BlockingLoreAPIClient` struct, which is responsible
 169 for managing requests, handling network communication (including headers, timeouts, etc.),
 170 and parsing XML responses from the site.

171 We implemented three primary endpoints to provide all the information patch-hub needs about
 172 patches from lore.kernel.org:

- 173 ▪ **request_available_lists:** returns all mailing lists of kernel subsystems available on
 174 lore.kernel.org. Listing 4 displays its implementation;
- 175 ▪ **request_patch_feed:** given a mailing list, returns the list of patchsets it contains;
- 176 ▪ **request_patch_html:** given a patchset ID, returns the complete information about it
 177 and its contents.

178 The integration module also includes another struct, `LoreSession`, which acts as an intermediary
 179 between the App screens and the external data source. It is responsible for calling the
 180 `BlockingLoreAPIClient`, processing XML responses, storing loaded patchsets along with their
 181 associated mailing lists, and providing this data to the App whenever required.

182 This design keeps the external data source decoupled from the core application logic, facilitating
 183 both testing and potential future replacement or extension of how patchsets are retrieved.

```

fn request_available_lists(&self, min_index: usize) -> Result<String, ClientError> {
    let available_lists_url = format!("{}/?&o={min_index}", self.lore_domain);

    let body: String = ureq::get(&available_lists_url)
        .header("Accept", "text/html,application/xhtml+xml,application/xml")
        .call()?
        .body_mut()
        .read_to_string()?;
    Ok(body)
}

```

184 **Listing 4:** Example of HTTP request to Lore.

185 Discussion

186 After detailing what patch-hub provides and how it is implemented, we next discuss less
 187 explicit aspects surrounding the project. First, we examine the motivation behind choosing
 188 Rust as the implementation language. Then, we discuss the importance of patch-hub, both

for its users and for its potential contributions to software engineering research. Finally, we outline the project's planned future directions.

Rust

There are two primary motivations behind choosing Rust for the development of patch-hub. First, although patch-hub does not have strict constraints such as high performance or limited memory usage, Rust offers several characteristics that provide universal benefits to software projects. Notably:

- Memory safety, enforced at compile time, which prevents a wide range of well-known programming bugs such as use-after-free, dangling pointers, and double free.
- Idiomatic expressiveness, arising from Rust's language design, which encourages clean code practices and enhances code readability. Key features include immutability by default and constructs such as Result, Option, and functional-style iterator combinators (e.g., map, filter, find, collect).

The second motivation is more abstract, directly related to the context in which patch-hub is situated and reflects recent trends in the Linux kernel development community. The adoption of Rust in patch-hub aligns with one of the project's implicit goals: modernizing the Linux kernel development process. In recent years, Rust's use in Linux has grown significantly. By the 2025 Linux Kernel Maintainer Summit, Rust was no longer considered experimental (Corbet, 2025). Linus Torvalds also expressed support for Rust in the kernel, clarifying that while maintainers are not required to adopt Rust in their own subsystems, they cannot block its use elsewhere (Torvalds, 2025). Although the kernel has historically been written in C, which provides high performance and a great deal of developer freedom, the motivation for incorporating Rust primarily lies in its compile-time memory safety guarantees, as mentioned above.

Thus, patch-hub follows this growing enthusiasm for the language and aligns itself with the community's ongoing trends. Moreover, contributing to patch-hub (or to any other open-source projects written in Rust) can be seen as an opportunity to prepare for future contributions to the kernel itself, especially considering that one of the key challenges in adopting Rust is its relatively steep learning curve.

Importance

A recurring concern among Linux kernel developers in recent years has been the sustainability of the development cycle, particularly given the bottlenecks created by the project's scale and outdated development processes. One possible way to address these challenges, as discussed in (Tadokoro et al., 2025a), is by increasing the use of development support tools, which can help reduce the cognitive and operational burden of tasks that are secondary to the system's evolution.

When interacting with patches, users must understand the dynamics of mailing lists and learn the steps and conventions involved in submitting and reviewing patches. These factors can slow down the development cycle and make it harder to integrate new contributors, reviewers, and maintainers.

patch-hub is one such support tool that aims to improve this scenario directly. By allowing users to visualize, validate, and respond to patchsets more quickly, intuitively, and in a centralized manner, the tool eliminates or simplifies many of the steps traditionally required in the process.

The lore.kernel.org platform itself is an example of a tool designed to simplify how users interact with patchsets. patch-hub builds on this well-established system, extending its functionality and usability so that users need nothing beyond their terminal to work with patchsets.

For these reasons, patch-hub can be viewed as a bridge between the traditional practices of kernel development, which depend on tools and technologies that are increasingly uncommon

in modern software engineering, and more contemporary approaches that emphasize user experience as a means to boost productivity and reduce the likelihood of errors. Furthermore, when considered within the broader context of its integration with kw, patch-hub can significantly expand the potential for automation and, consequently, accelerate the entire development workflow.

Another point worth highlighting is the tool's potential to serve as a platform for experimentation and metrics collection regarding the kernel contribution process. The analysis of data and feedback generated during its use could enable investigations into different aspects of the patch review cycle, such as review time, volume, and engagement in reviews, among other metrics related to reviewers' interactions with patches.

In this way, patch-hub not only facilitates the daily work of contributors but also creates opportunities for comparative studies between its use and the traditional patch review flow, fostering broader discussions about collaboration and efficiency in large-scale projects, and specifically how these factors can impact the future of Linux kernel development.

Next steps

There are two clear next steps for patch-hub. The first is to improve the tool's integration with its predecessor, kw, so that it becomes possible, for example, to compile and deploy a patch or patchset under review in a more automated way, directly from patch-hub itself. Furthermore, given the strong relationship between the tools, additional initiatives can be undertaken to enhance this integration, making the overall development flow even more centralized and seamless.

The second step involves instrumenting patch-hub to enable, with user consent, the collection of telemetry data during its execution. By anonymizing and sending this information to a server, it would be possible to consolidate user data and analyze it to identify bottlenecks, understand usage patterns, and propose improvements that reduce friction in the revision flow. The existence of this metrics collection infrastructure will facilitate the design and comparison of future experiments involving Linux kernel developers. The existence of this metrics collection infrastructure will also facilitate the design and comparison of future experiments involving Linux kernel developers. Finally, it can support broader reflections on tool usage and user behavior, as discussed in the previous section.

Acknowledgements

This study was financed by CAPES (Finance Code 001), the São Paulo Research Foundation – FAPESP (Proc. 2025/05395-0) and the São Paulo State Data Analysis System Foundation – SEADE (FAPESP Proc. 2023/18026-8), Brazil.

References

- Corbet, J. (2013). *On saying "no"*. Linux Weekly News. <https://lwn.net/Articles/571995/>
- Corbet, J. (2018). *Two perspectives on the maintainer relationship*. Linux Weekly News. <https://lwn.net/Articles/749676/>
- Corbet, J. (2021). *Maintainers truth and fiction*. Linux Weekly News. <https://lwn.net/Articles/842415/>
- Corbet, J. (2025). *The (successful) end of the kernel rust experiment*. Linux Weekly News. <https://lwn.net/Articles/1049831/>
- Dean, S. (2020). *The maintainer's paradox: Balancing project and community*. Linux Foundation. <https://www.linuxfoundation.org/blog/the-maintainers-paradox-balancing-project-and-community>

- 280 Edge, J. (2013). *The kernel maintainer gap*. Linux Weekly News. [https://lwn.net/Articles/](https://lwn.net/Articles/572003/)
281 [572003/](https://lwn.net/Articles/572003/)
- 282 Edge, J. (2016). *On moving on from being a maintainer*. Linux Weekly News. [https:](https://lwn.net/Articles/670088/)
283 [//lwn.net/Articles/670088/](https://lwn.net/Articles/670088/)
- 284 Edge, J. (2018). *Too many lords, not enough stewards*. Linux Weekly News. [https://lwn.net/](https://lwn.net/Articles/745817/)
285 [Articles/745817/](https://lwn.net/Articles/745817/)
- 286 Rigby, P. C., German, D. M., Cowen, L., & Storey, M.-A. (2014). Peer review on open-source
287 software projects: Parameters, statistical models, and theory. *ACM Trans. Softw. Eng.*
288 *Methodol.*, 23(4). <https://doi.org/10.1145/2594458>
- 289 Tadokoro, D., Siqueira, R., & Meirelles, P. (2025a). Can the linux kernel sustain 30 more years
290 of growth? Toward mitigating bottlenecks in its development model. *Anais Do XXXIX*
291 *Simpósio Brasileiro de Engenharia de Software*, 845–851. [https://doi.org/10.5753/sbes.](https://doi.org/10.5753/sbes.2025.11607)
292 [2025.11607](https://doi.org/10.5753/sbes.2025.11607)
- 293 Tadokoro, D., Siqueira, R., & Meirelles, P. (2025b). Kworkflow a linux kernel developer
294 automation workflow system. *Anais Do XXXIX Simpósio Brasileiro de Engenharia de*
295 *Software*, 949–955. <https://doi.org/10.5753/sbes.2025.11322>
- 296 Torvalds, L. (2025). *Re: Rust kernel policy*. Email message sent to the [https:](https://lore.kernel.org/rust-for-linux)
297 [//lore.kernel.org/rust-for-linux](https://lore.kernel.org/rust-for-linux) mailing list. [https://lore.kernel.org/rust-for-linux/CAHk=](https://lore.kernel.org/rust-for-linux/CAHk=wgLbz1Bm8QhmJ4dJGSmTuV5w_R0Gwvg5kHrYr4Ko9dUHQ@mail.gmail.com/T/#u)
298 [wgLbz1Bm8QhmJ4dJGSmTuV5w_R0Gwvg5kHrYr4Ko9dUHQ@mail.gmail.com/T/#u](https://lore.kernel.org/rust-for-linux/CAHk=wgLbz1Bm8QhmJ4dJGSmTuV5w_R0Gwvg5kHrYr4Ko9dUHQ@mail.gmail.com/T/#u)
- 299 Vetter, D. (2017). *Maintainers don't scale*. [https://blog.ffwll.ch/2017/01/maintainers-dont-scale.](https://blog.ffwll.ch/2017/01/maintainers-dont-scale.html)
300 [html](https://blog.ffwll.ch/2017/01/maintainers-dont-scale.html)